სტრინგები, მონაცემთა სტრუქტურები და ციკლები

ნაწილი	რას მოიცავს
01 · String Methods	8 მეთოდი, არგუმენტები, დაბრუნებული შედეგები და chaining
02 · მასივები	ინდექსები, slicing, მეთოდები, len(), in/not in
03 · Tuples	უცვლელობა, indexing, count(), index()
04 · Sets	უნიკალურობა, შეცვლა და 4 ძირითადი ოპერაცია
05 · While Loop	პირობა, იტერაცია, break, continue

ნაწილი 01 / სტრინგის მეთოდები

String (str) არის ტექსტური მნიშვნელობა. სტრინგის მეთოდები გვაძლევს საშუალებას შევცვალოთ ასოების ფორმა, გავასუფთაოთ ტექსტი, ვიპოვოთ ინფორმაცია ან დავყოთ ტექსტი ნაწილებად.

1.1 რას ნიშნავს მეთოდი?

მეთოდი არის მოქმედება, რომელსაც კონკრეტული ტიპის მნიშვნელობაზე ვიძახებთ წერტილის (.) გამოყენებით. მაგალითად, **name.lower()** იძახებს lower მეთოდს name სტრინგზე.

```
name = "NIKA"
result = name.lower()
print(result)  # nika
```

ფუნქცია და მეთოდი: print(name) არის ფუნქციის გამოძახება; **name.upper()** კი მეთოდის გამოძახებაა.

არგუმენტი: დამატებითი ინფორმაცია ფრჩხილებში. მაგალითად, **text.replace("a", "b")**-ში ორი არგუმენტია.

დაბრუნებული მნიშვნელობა: ის შედეგი, რომელსაც მეთოდი გვაძლევს და შეგვიძლია ცვლადში შევინახოთ.

> **გახსოვდეთ:** სტრინგები უცვლელია (immutable): name.upper() თავდაპირველ name ცვლადს არ ცვლის. შესანახად გამოიყენეთ name = name.upper().

1.2 lower()

დანიშნულება: ყველა ასოს გარდაქმნის პატარა ასოდ; გამოსადეგია მომხმარებლის პასუხების შედარებისა და ტექსტის ერთნაირ ფორმატში გადაყვანისთვის.

სინტაქსი: text.lower()

არგუმენტები: არ იღებს არგუმენტებს.

შედეგი: აბრუნებს ახალ სტრინგს.

```
answer = "  YES  "
print(answer.strip().lower() == "yes")  # True
```

> **გახსოვდეთ:** დიდი/პატარა ასოების მიმართ განსხვავება ქრება მხოლოდ იმ ახალ შედეგში, რომელზეც lower() გამოვიყენეთ.

1.3 upper()

დანიშნულება: სტრინგში ასოებს გარდაქმნის დიდ ასოებად. გამოიყენება სათაურების, კოდებისა და სტანდარტული ფორმატის გამოსატანად.

სინტაქსი: **text.upper()**

არგუმენტები: არ იღებს არგუმენტებს.

შედეგი: აბრუნებს ახალ სტრინგს.

```
text = "Hello 123!"
print(text.upper())  # HELLO 123!
```

> **გახსოვდეთ:** ციფრები და ჩვეულებრივი პუნქტუაცია არ იცვლება.

1.4 capitalize()

დანიშნულება: პირველ სიმბოლოს დიდ ასოდ გარდაქმნის (თუ ასოა), ხოლო დანარჩენ ასოებს აპატარავებს.

სინტაქსი: **text.capitalize()**

არგუმენტები: არ იღებს არგუმენტებს.

შედეგი: აბრუნებს ახალ სტრინგს.

```
text = "hELLO PYTHON"
print(text.capitalize())  # Hello python
```

> **გახსოვდეთ:** ეს არ არის იგივე, რაც title(): capitalize() მხოლოდ სტრინგის დასაწყისის ასოს ადიდებს.

1.5 title()

დანიშნულება: ტექსტს სათაურის მსგავს ფორმაში გარდაქმნის: სიტყვების პირველი ასოები დიდი ხდება, დანარჩენი — პატარა.

სინტაქსი: **text.title()**

არგუმენტები: არ იღებს არგუმენტებს.

შედეგი: აბრუნებს ახალ სტრინგს.

```
text = "hello PYTHON world"
print(text.title())  # Hello Python World
```

> **გახსოვდეთ:** სიტყვების საზღვრები ყოველთვის ბუნებრივი ენის წესებით არ განისაზღვრება: მაგალითად, ინგლისურ აპოსტროფებთან შედეგი ზოგჯერ მოულოდნელია.

1.6 strip()

დანიშნულება: სტრინგის დასაწყისიდან და ბოლოდან შლის whitespace სიმბოლოებს ან მითითებულ სიმბოლოებს.

სინტაქსი: text.strip(chars=None)

არგუმენტები: არასავალდებულო **chars**: მოსაშორებელი სიმბოლოების ერთობლიობა. არგუმენტის გარეშე იშლება გარე გამოტოვებები, tab-ები და ახალი ხაზის სიმბოლოები.

შედეგი: აბრუნებს ახალ სტრინგს.

```
text = "!!! Hello !!"
print(text.strip("! "))  # Hello
print("  A   B  ".strip())  # A   B
```

> **გახსოვდეთ:** strip("!?") არ ეძებს მთლიან სიტყვას "!?"; ის კიდეებიდან ნებისმიერი რაოდენობით შლის სიმბოლოებს ! და ?. ტექსტის შუაში არაფერს ცვლის.

1.7 replace()

დანიშნულება: სტრინგის მითითებულ ნაწილს ახალი ტექსტით ანაცვლებს.

სინტაქსი: text.replace(old, new, count=-1)

არგუმენტები: old — ძველი ტექსტი (აუცილებელი); **new** — ახალი ტექსტი (აუცილებელი); **count** — რამდენი ჩანაცვლება შესრულდეს მაქსიმუმ (არასავალდებულო).

შედეგი: აბრუნებს ახალ სტრინგს.

```
text = "a-a-a"
print(text.replace("a", "b", 2))  # b-b-a
```

> **გახსოვდეთ:** მეთოდი არჩევს დიდ/პატარა ასოებს: "Python" და "python" განსხვავებული ტექსტებია.

1.8 count()

დანიშნულება: ითვლის, რამდენჯერ გვხვდება მითითებული სტრინგი სხვა სტრინგში.

სინტაქსი: text.count(sub, start=0, end=...)

არგუმენტები: sub — მოსაძებნი ტექსტი (აუცილებელი); **start** — საწყისი ინდექსი და **end** — საბოლოო საზღვარი (არასავალდებულო). end არ შედის შესამოწმებელ მონაკვეთში.

შედეგი: აბრუნებს მთელ რიცხვს (int).

```
text = "banana"
print(text.count("a"))       # 3
print(text.count("a", 0, 3)) # 1
```

გახსოვდეთ: count() ასხვავებს ასოების ზომებს; ყველა ვარიანტის ერთად დასათვლელად ჯერ lower() გამოიყენეთ. თანმთხვევები არ გადაიფარება.

1.9 split()

დანიშნულება: სტრინგს ყოფს ნაწილებად და ქმნის მასივს (list), რომლის ელემენტებია გამოყოფილი ნაწილები.

სინტაქსი: text.split(sep=None, maxsplit=-1)

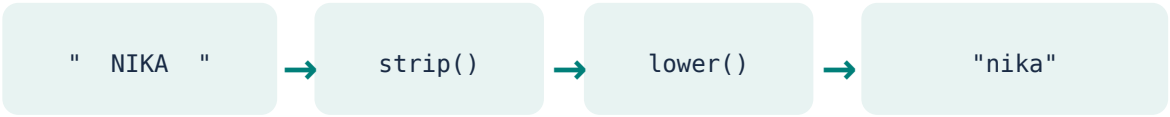
არგუმენტები: sep — გამყოფი ტექსტი (არასავალდებულო); **maxsplit** — დაშვების მაქსიმალური რაოდენობა (არასავალდებულო).

შედეგი: აბრუნებს მასივს (list), არა სტრინგს.

```
print("red  blue".split())
# ["red", "blue"]
print("a,b,c".split(",", 1))
# ["a", "b,c"]
```

გახსოვდეთ: split() არგუმენტის გარეშე გამოყოფს სიტყვებს whitespace-ით და ზედმეტ გამოტოვებებს გამოტოვებს; split(",") იყენებს ზუსტად მძიმეს გამყოფად.

1.10 Method chaining — მეთოდების მიყოლებით გამოყენება



ყოველი მეთოდის შედეგი გადაეცემა შემდეგ მეთოდს.

```
name = "  NIKA  "
clean_name = name.strip().lower().capitalize()
print(clean_name)  # Nika
```

გახსოვდეთ: წერტილის შემდეგ მეთოდი იძახება წინა ოპერაციის შედეგზე. split() აბრუნებს მასივს, ამიტომ მის შემდეგ ყველა სტრინგის მეთოდს ვეღარ გამოვიძახებთ.

1.11 მეთოდების სწრაფი შედარება

მეთოდი	აუცილებელი არგუმენტი	რა აბრუნებს
lower(), upper(), capitalize(), title()	არცერთი	ახალი str
strip(chars)	არცერთი; chars არასავალდებულოა	ახალი str
replace(old, new, count)	old, new	ახალი str
count(sub, start, end)	sub	int
split(sep, maxsplit)	არცერთი	list

ნაწილი 02 / მასივები (Lists)

მასივი (list) გამოიყენება ერთ ცვლადში მრავალი მნიშვნელობის შესანახად. ელემენტებს აქვთ რიგი და ინდექსები, ხოლო მასივის შეცვლა შესაძლებელია.

2.1 შექმნა და ძირითადი თვისებები

```
students = ["Ana", "Nika", "Ana"]
numbers = [10, 20, 30]
mixed = ["Nika", 19, True]
empty = []
```

- **დალაგებულია** — ელემენტების რიგი შენარჩუნებულია.
- **ცვალებადია (mutable)** — შესაძლებელია ელემენტის დამატება, შეცვლა და წაშლა.
- **დუბლიკატები დაშვებულია** — ერთი და იგივე მნიშვნელობა შეიძლება განმეორდეს.
- შეიძლება შეინახოს სხვადასხვა ტიპის მნიშვნელობები.

2.2 ინდექსები და slicing

ინდექსი ყოველთვის 0-დან იწყება

"Ana"	"Nika"	"Giorgi"	"Mariam"
0	1	2	3

students[0] → "Ana" students[-1] → "Mariam"

```
students = ["Ana", "Nika", "Giorgi", "Mariam"]
print(students[0])    # Ana
print(students[-1])   # Mariam
print(students[1:3])  # ["Nika", "Giorgi"]
students[0] = "Luka"  # პირველი ელემენტის შეცვლა
```

ინდექსი: ელემენტის პოზიცია იწყება 0-დან; **-1** ნიშნავს ბოლო ელემენტს. **Slicing** გამოიყენება მონაკვეთის მისაღებად: **items[start:stop:step]**; stop ინდექსი შედეგში არ შედის.

> **გახსოვდეთ:** თუ არსებობს 4 ელემენტი, სწორი დადებითი ინდექსებია 0, 1, 2, 3. items[4] გამოიწვევს IndexError-ს. slicing კი აბრუნებს ახალ მასივს.

2.3 len(), in / not in და for ციკლი

```
students = ["Ana", "Nika", "Mariam"]
print(len(students))         # 3
print("Nika" in students)    # True
print("Luka" not in students) # True
for student in students:
    print(student)
```

len(items) არის ფუნქცია, იღებს ერთ კოლექციას და აბრუნებს ელემენტების რაოდენობას. **in** / **not in** წევრობის ოპერატორებია და აბრუნებენ True ან False-ს. **for** საშუალებას გვაძლევს თითოეული ელემენტი ცალ-ცალკე დავამუშაოთ.

2.4 append()

დანიშნულება: მასივის ბოლოში ამატებს ზუსტად ერთ ელემენტს.

სინტაქსი: items.append(value)

არგუმენტები: value — ერთი დასამატებელი მნიშვნელობა (აუცილებელი).

შედეგი: მასივს ცვლის პირდაპირ; აბრუნებს None-ს.

```
fruits = ["apple"]
fruits.append("pear")
print(fruits)  # ["apple", "pear"]
```

> **გახსოვდეთ:** თუ value თავად მასივია, ის მთლიანად ერთ ელემენტად დაემატება.

2.5 insert()

დანიშნულება: ახალ ელემენტს ათავსებს მითითებულ ინდექსზე და შემდეგ ელემენტებს მარჯვნივ წევს.

სინტაქსი: items.insert(index, value)

არგუმენტები: index — ჩასმის პოზიცია; **value** — დასამატებელი მნიშვნელობა. ორივე აუცილებელია.

შედეგი: მასივს ცვლის პირდაპირ; აბრუნებს None-ს.

```
nums = [10, 30]
nums.insert(1, 20)
print(nums)  # [10, 20, 30]
```

> **გახსოვდეთ:** ეს არ არის არსებული ელემენტის ჩანაცვლება: ელემენტების რაოდენობა იზრდება.

2.6 extend()

დანიშნულება: მასივს ამატებს სხვა iterable-ის (მაგალითად, მეორე მასივის) ყველა ელემენტს ცალ-ცალკე.

სინტაქსი: items.extend(iterable)

არგუმენტები: iterable — კოლექცია/მიმდევრობა, რომლის ელემენტებიც უნდა დაემატოს (აუცილებელი).

შედეგი: მასივს ცვლის პირდაპირ; აბრუნებს None-ს.

```
nums = [1, 2]
nums.extend([3, 4])
print(nums)  # [1, 2, 3, 4]
```

> **გახსოვდეთ:** append([3,4]) ამატებს ერთ შიდა მასივს, ხოლო extend([3,4]) ამატებს ორ ცალკე რიცხვს.

2.7 remove()

დანიშნულება: სიიდან შლის მითითებული მნიშვნელობის პირველ შეხვედრას.

სინტაქსი: items.remove(value)

არგუმენტები: value — მოსაშორებელი მნიშვნელობა (აუცილებელი).

შედეგი: მასივს ცვლის პირდაპირ; აბრუნებს None-ს.

```
nums = [2, 3, 2]
nums.remove(2)
print(nums)  # [3, 2]
```

> **გახსოვდეთ:** თუ მნიშვნელობა მასივში არ არის, წარმოიქმნება ValueError.

2.8 pop()

დანიშნულება: შლის მითითებული ინდექსის ელემენტს; ინდექსის გარეშე შლის ბოლო ელემენტს.

სინტაქსი: items.pop(index=-1)

არგუმენტები: არასავალდებულო **index** — მოსაშორებელი ელემენტის ინდექსი.

შედეგი: მასივს ცვლის და აბრუნებს წაშლილ ელემენტს.

```
nums = [10, 20, 30]
removed = nums.pop()
print(removed)  # 30
print(nums)     # [10, 20]
```

> **გახსოვდეთ:** pop() განსხვავდება remove()-სგან: pop() მუშაობს ინდექსით და შედეგსაც აბრუნებს. ცარიელ მასივზე გამოიწვევს IndexError-ს.

2.9 clear()

დანიშნულება: სიიდან ყველა ელემენტს შლის.

სინტაქსი: items.clear()

არგუმენტები: არ იღებს არგუმენტებს.

შედეგი: ცვლის მასივს; აბრუნებს None-ს.

```
names = ["Ana", "Nika"]
names.clear()
print(names)  # []
```

> **გახსოვდეთ:** მასივი კვლავ არსებობს, უბრალოდ ცარიელი ხდება: [].

2.10 count()

დანიშნულება: ითვლის, რამდენჯერ გვხვდება კონკრეტული მნიშვნელობა მასივში.

სინტაქსი: items.count(value)

არგუმენტები: value — საძიებელი მნიშვნელობა (აუცილებელი).

შედეგი: აბრუნებს მთელ რიცხვს (int), მასივს არ ცვლის.

```
numbers = [1, 2, 2, 3]
print(numbers.count(2))  # 2
```

> **გახსოვდეთ:** ეს მასივის მეთოდია; სტრინგის count()-ისგან განსხვავებით, ითვლის ელემენტებს და არა ტექსტის ნაწილებს.

2.11 index()

დანიშნულება: პოულობს მნიშვნელობის პირველი შეხვედრის ინდექსს.

სინტაქსი: items.index(value, start=0, stop=...)

არგუმენტები: value — საძიებელი მნიშვნელობა (აუცილებელი); **start** და **stop** — ძებნის საზღვრები (არასავალდებულო).

შედეგი: აბრუნებს ინდექსს (int).

```
fruits = ["apple", "pear", "apple"]
print(fruits.index("apple"))  # 0
```

> **გახსოვდეთ:** თუ ელემენტი არ მოიძებნა, წარმოიქმნება ValueError. საჭიროებისას ჯერ შეამოწმეთ value in items.

2.12 sort()

დანიშნულება: მასივის ელემენტებს ახარისხებს ზრდადობით ან მითითებული წესით.

სინტაქსი: items.sort(key=None, reverse=False)

არგუმენტები: key — შედარების წესის ფუნქცია (არასავალდებულო, ამ დონეზე გამოტოვეთ); **reverse** — False/True, შებრუნებული რიგისთვის.

შედეგი: მასივს ცვლის პირდაპირ; აბრუნებს None-ს.

```
nums = [3, 1, 2]
nums.sort()
print(nums)  # [1, 2, 3]
nums.sort(reverse=True)
print(nums)  # [3, 2, 1]
```

გახსოვდეთ: მარტივ მაგალითებში დაახარისხეთ ერთმანეთთან თავსებადი მნიშვნელობები, მაგალითად მხოლოდ რიცხვები ან მხოლოდ სტრინგები.

2.13 reverse()

დანიშნულება: ელემენტების არსებულ რიგს აბრუნებს უკუღმა.

სინტაქსი: items.reverse()

არგუმენტები: არ იღებს არგუმენტებს.

შედეგი: მასივს ცვლის პირდაპირ; აბრუნებს None-ს.

```
nums = [5, 1, 8]
nums.reverse()
print(nums)  # [8, 1, 5]
```

გახსოვდეთ: reverse() არ არის დახარისხება: იგი მხოლოდ უკვე არსებული თანმიმდევრობის შებრუნებაა.

2.14 მასივების ყველაზე მნიშვნელოვანი წესები

- **append / insert / extend / remove / pop / clear / sort / reverse** ცვლიან არსებულ მასივს.
- ჩვეულებრივ, მასივის შემცვლელი მეთოდები აბრუნებენ None-ს; გამონაკლისია **pop()**, რომელიც წაშლილ ელემენტს აბრუნებს.
- **count()** ითვლის მნიშვნელობის შეხვედრებს, **index()** პოულობს მის პირველ პოზიციას.
- **in** ამოწმებს არსებობას; **len()** ითვლის ელემენტებს; არცერთი არ ცვლის მასივს.

ყურადღება: არ დაწეროთ numbers = numbers.sort(): sort() აბრუნებს None-ს, რის შედეგადაც numbers დაკარგავს მასივს.

ნაწილი 03 / Tuple — უცვლელი მიმდევრობა

Tuple ჰგავს მასივს, მაგრამ მისი ელემენტების დამატება, წაშლა და პირდაპირი ჩანაცვლება შეუძლებელია. იგი სასარგებლოა ფიქსირებული მონაცემებისთვის.

3.1 შექმნა, ინდექსები და slicing

```
point = (10, 20)
colors = ("red", "blue", "red")
single = ("red",)  # ერთი ელემენტი
empty = ()
print(point[0])    # 10
print(colors[-1]) # red
print(colors[0:2]) # ("red", "blue")
```

თვისებები: tuple დალაგებულია, ინარჩუნებს დუბლიკატებს და მხარს უჭერს ინდექსს, slicing-სა და for ციკლს. განსხვავება მასივთან არის **უცვლელობა (immutable)**.

> **გახსოვდეთ:** ერთელემენტიანი tuple-ისთვის აუცილებელია მძიმე: ("red",). მხოლოდ ("red") არის ჩვეულებრივი სტრინგი ფრჩხილებში.

3.2 Tuple-ის მეთოდები

3.2.1 count()

დანიშნულება: ითვლის, რამდენჯერ გვხვდება მოცემული მნიშვნელობა tuple-ში.

სინტაქსი: **values.count(value)**

არგუმენტები: value — საძიებელი მნიშვნელობა (აუცილებელი).

შედეგი: აბრუნებს int-ს; tuple უცვლელი რჩება.

```
colors = ("red", "blue", "red")
print(colors.count("red"))  # 2
```

> **გახსოვდეთ:** მეთოდი ძალიან ჰგავს list.count()-ს.

3.2.2 index()

დანიშნულება: აბრუნებს საძიებელი მნიშვნელობის პირველ ინდექსს.

სინტაქსი: **values.index(value, start=0, stop=...)**

არგუმენტები: **value** აუცილებელია; **start** და **stop** არასავალდებულო საზღვრებია.

შედეგი: აბრუნებს int-ს; tuple უცვლელი რჩება.

```
values = (10, 20, 30, 20)
print(values.index(20))  # 1
```

> **გახსოვდეთ:** თუ მნიშვნელობა ვერ მოიძებნა, წარმოიქმნება ValueError.

3.3 len(), in, not in და for

```
days = ("Mon", "Tue", "Wed")
print(len(days))          # 3
print("Tue" in days)     # True
print("Sun" not in days) # True
for day in days:
    print(day)
```

> **გახსოვდეთ:** tuple[0] = value შეცდომას გამოიწვევს. Tuple გამოიყენეთ, როცა მონაცემების რიგი მნიშვნელოვანია, მაგრამ ელემენტების შეცვლა არ არის საჭირო.

ნაწილი 04 / Set — უნიკალური ელემენტების კოლექცია

Set ინახავს უნიკალურ ელემენტებს. მას არ აქვს განსაზღვრული ინდექსები და არ არის განკუთვნილი ელემენტების კონკრეტული რიგით დასამუშავებლად.

4.1 შექმნა და თვისებები

```
numbers = {1, 2, 2, 3}
print(len(numbers))  # 3
empty = set()      # ცარიელი set
not_a_set = {}     # ცარიელი dictionary
```

- დუბლიკატები ავტომატურად იშლება.
- ელემენტების ჩვენების რიგი გარანტირებული არ არის.
- Set-ის შეცვლა შესაძლებელია, მაგრამ ელემენტებზე **[0]** ინდექსით ვერ მივმართავთ.
- ელემენტები უნდა იყოს hashable ტიპის; მარტივ მაგალითებში გამოიყენეთ რიცხვები, სტრინგები ან შესაბამისი tuple-ები.

4.2 ელემენტების დამატება და წაშლა

4.2.1 add()

დანიშნულება: Set-ში ამატებს ერთ ელემენტს; არსებული დუბლიკატი დამატებით ასლს არ ქმნის.

სინტაქსი: **items.add(value)**

არგუმენტები: value — ერთი ელემენტი (აუცილებელი).

შედეგი: ცვლის set-ს; აბრუნებს None-ს.

```
fruits = {"apple", "pear"}
fruits.add("apple")
fruits.add("kiwi")
print(len(fruits))  # 3
```

> **გახსოვდეთ:** მასივის append()-ისგან განსხვავებით, set-ში განმეორებითი მნიშვნელობები არ ინახება.

4.2.2 update()

დანიშნულება: Set-ში ამატებს ერთი ან რამდენიმე iterable-ის ყველა ელემენტს.

სინტაქსი: items.update(iterable1, iterable2, ...)

არგუმენტები: შეუძლია მიიღოს ერთი ან რამდენიმე iterable (მაგალითად მასივი ან set); არგუმენტის გარეშეც მოქმედებს, მაგრამ არაფერს ცვლის.

შედეგი: ცვლის set-ს; აბრუნებს None-ს.

```
nums = {1, 2}
nums.update([2, 3, 4])
print(len(nums))  # 4
```

> **გახსოვდეთ:** update([3,4]) ამატებს 3-სა და 4-ს ცალ-ცალკე.

4.2.3 remove()

დანიშნულება: Set-იდან შლის მითითებულ ელემენტს.

სინტაქსი: items.remove(value)

არგუმენტები: value — მოსაშორებელი ელემენტი (აუცილებელი).

შედეგი: ცვლის set-ს; აბრუნებს None-ს.

```
fruits = {"apple", "pear"}
fruits.remove("pear")
```

> **გახსოვდეთ:** თუ ასეთი ელემენტი არ არსებობს, წარმოიქმნება KeyError.

4.2.4 discard()

დანიშნულება: Set-იდან შლის მითითებულ ელემენტს, თუ ის არსებობს.

სინტაქსი: items.discard(value)

არგუმენტები: value — მოსაშორებელი ელემენტი (აუცილებელი).

შედეგი: ცვლის set-ს; აბრუნებს None-ს.

```
fruits = {"apple"}
fruits.discard("pear")  # შეცდომის გარეშე
```

> **გახსოვდეთ:** remove()-ისგან განსხვავებით, არარსებული ელემენტის გამო შეცდომას არ იწვევს.

4.2.5 pop()

დანიშნულება: Set-იდან შლის და აბრუნებს ერთ ნებისმიერ ელემენტს.

სინტაქსი: **items.pop()**

არგუმენტები: არ იღებს არგუმენტებს.

შედეგი: ცვლის set-ს და აბრუნებს ამოღებულ ელემენტს.

```
values = {10, 20}
removed = values.pop()
print(removed)  # 10 ან 20
```

> **გახსოვდეთ:** არ იძლევა გარანტიას, რომელი ელემენტი წაიშლება. ცარიელ set-ზე გამოიწვევს KeyError-ს.

4.2.6 clear()

დანიშნულება: Set-იდან ყველა ელემენტს შლის.

სინტაქსი: **items.clear()**

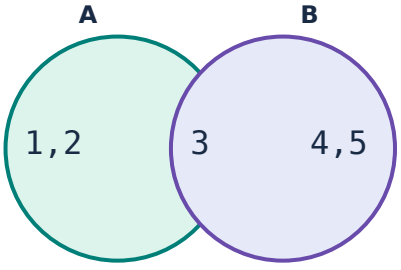
არგუმენტები: არ იღებს არგუმენტებს.

შედეგი: ცვლის set-ს; აბრუნებს None-ს.

```
values = {1, 2, 3}
values.clear()
print(values)  # set()
```

> **გახსოვდეთ:** კოლექცია ცარიელდება, მაგრამ ცვლადი რჩება set ტიპის.

4.3 Set ოპერაციები — საერთო და განსხვავებული ელემენტები



მარცხნივ მხოლოდ A; შუაში საერთო ელემენტები; მარჯვნივ მხოლოდ B.

```
a = {1, 2, 3}
b = {3, 4, 5}
```

ქვემოთ მოცემული მეთოდები აბრუნებენ **ახალ set-ს**. ისინი საწყის a და b კოლექციებს არ ცვლიან.

4.3.1 union()

დანიშნულება: აერთიანებს ორივე set-ის უნიკალურ ელემენტებს.

სინტაქსი: **a.union(b) ან a | b**

არგუმენტები: შეიძლება მიიღოს ნული ან რამდენიმე iterable; მაგალითში **b** მეორე set-ია.

შედეგი: ახალი set.

```
a = {1, 2, 3}; b = {3, 4, 5}
print(a | b)  # {1, 2, 3, 4, 5}
```

> **გახსოვდეთ:** შედეგში დუბლიკატები არ ინახება.

4.3.2 intersection()

დანიშნულება: ტოვებს მხოლოდ იმ ელემენტებს, რომლებიც ორივე set-ში გვხვდება.

სინტაქსი: **a.intersection(b) ან a & b**

არგუმენტები: შეიძლება მიიღოს ნული ან რამდენიმე iterable; მაგალითში **b** მეორე set-ია.

შედეგი: ახალი set.

```
a = {1, 2, 3}; b = {3, 4, 5}
print(a & b)  # {3}
```

> **გახსოვდეთ:** ეს არის ორი ჯგუფის საერთო ნაწილი.

4.3.3 difference()

დანიშნულება: აბრუნებს A-ში არსებულ იმ ელემენტებს, რომლებიც B-ში არ არის.

სინტაქსი: **a.difference(b) ან a - b**

არგუმენტები: შეიძლება მიიღოს ნული ან რამდენიმე iterable; მაგალითში **b** მეორე set-ია.

შედეგი: ახალი set.

```
a = {1, 2, 3}; b = {3, 4, 5}
print(a - b)  # {1, 2}
```

> **გახსოვდეთ:** მიმართულებას მნიშვნელობა აქვს: a - b და b - a ჩვეულებრივ განსხვავდება.

4.3.4 symmetric_difference()

დანიშნულება: აბრუნებს ელემენტებს, რომლებიც მხოლოდ ერთ-ერთ set-შია და არა ორივეში.

სინტაქსი: **a.symmetric_difference(b) ან a ^ b**

არგუმენტები: b — მეორე set ან iterable; აუცილებელი არგუმენტი.

შედეგი: ახალი set.

```
a = {1, 2, 3}; b = {3, 4, 5}
print(a ^ b)  # {1, 2, 4, 5}
```

> **გახსოვდეთ:** საერთო ელემენტები შედეგიდან გამოირიცხება.

4.4 len(), in და for set-ზე

```
skills = {"Python", "Git", "SQL"}
print(len(skills))            # 3
print("Git" in skills)       # True
for skill in skills:
    print(skill)  # რიგი არ არის გარანტირებული
```

> **გახსოვდეთ:** Set გამოიყენეთ უნიკალური მნიშვნელობებისა და წევრობის შემოწმებისთვის. თუ რიგი ან ინდექსებით წვდომაა საჭირო, გამოიყენეთ list ან tuple.

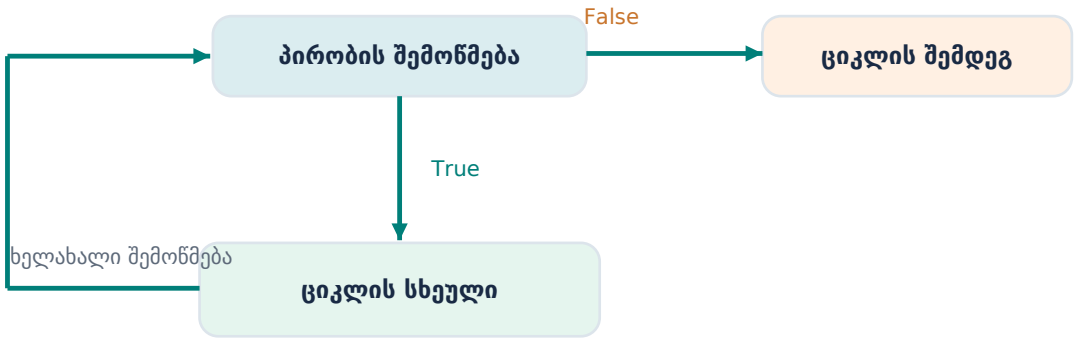
ნაწილი 05 / While Loop — პირობითი გამეორება

while გამოიყენება ერთი და იმავე მოქმედების გასამეორებლად მანამ, სანამ პირობა True-ა. განსაკუთრებით გამოსადეგია, როცა წინასწარ არ ვიცით რამდენი გამეორება დაგვჭირდება.

5.1 სინტაქსი და ძირითადი მუშაობა

```
while condition:
    # მოქმედებები, სანამ condition არის True
    statement
```

condition არის პირობა, რომელიც ყოველ იტერაციამდე მოწმდება. **:** აუცილებელია; შიგნით არსებული კოდი ინდენტაციით იწერება. როდესაც პირობა False ხდება, ციკლი სრულდება.



თუ თავიდანვე False-ია, ციკლის სხეული არც ერთხელ არ სრულდება.

5.2 მთვლელით მართვა

დანიშნულება: ცვლადის მნიშვნელობა ყოველ გამეორებაზე იცვლება, რათა ბოლოს პირობა False გახდეს.

სინტაქსი: while number <= 5:

არგუმენტები: while-ს ფრჩხილებში მეთოდის მსგავსი არგუმენტები არ აქვს; ის ამოწმებს მის შემდეგ დაწერილ ლოგიკურ პირობას.

შედეგი: შედეგი არის განმეორებითი შესრულება, არა მეთოდის დაბრუნებული მნიშვნელობა.

```
number = 1
while number <= 5:
    print(number)
    number += 1
```

გახსოვდეთ: თუ number არ გაიზრდება, ციკლი შეიძლება უსასრულოდ გაგრძელდეს.

5.3 for და while — როდის რომელი?

for	while
გამოსადეგია კოლექციის ელემენტების გასავლელად ან გამეორებების ცნობილი რაოდენობისას.	გამოსადეგია, როდესაც გამეორება პირობის შესრულებაზეა დამოკიდებული.
for item in items:	while condition:

5.4 while True და break

while True ქმნის ციკლს, რომლის პირობა მუდმივად True-ა. გასასვლელად შეგვიძლია გამოვიყენოთ **break**, რომელიც მაშინვე ასრულებს მიმდინარე ციკლს.

```
while True:
    password = input("Password: ")
    if password == "1234":
        print("Welcome")
        break
    print("Try again")
```

გახსოვდეთ: break მუშაობს while-შიც და for-შიც. ის ასრულებს უახლოეს იმ ციკლს, რომელშიც შესრულდა.

5.5 continue — მიმდინარე იტერაციის გამოტოვება

continue არ ასრულებს მთლიან ციკლს; ის გამოტოვებს მიმდინარე იტერაციის დარჩენილ კოდს და გადადის შემდეგი შემოწმებისკენ.

```
number = 0
while number < 6:
    number += 1
    if number == 3:
        continue
    print(number)  # 1, 2, 4, 5, 6
```

ყურადღება: while-ში მრიცხველის ცვლილება continue-მდე ჩაწერეთ, თუ მისი გამოტოვება ციკლს უსასრულოს გახდის.

5.6 რა განსხვავებაა break-სა და continue-ს შორის?

კონსტრუქცია	რა ხდება	შემდეგი ნაბიჯი
break	მიმდინარე ციკლი სრულდება	სრულდება კოდი ციკლის შემდეგ
continue	მიმდინარე გამეორების დარჩენილი ნაწილი გამოტოვებულია	while-ის პირობა ხელახლა მოწმდება

ნაწილი 06 / შეჯამება — რა უნდა გახსოვდეთ?

მოკლე შედარება და სინტაქსის საცნობარო ცხრილები, რათა მასალის გამეორება სწრაფად შეძლოთ.

6.1 მონაცემთა სტრუქტურების შედარება

თვისება	მასივი [ ]	Tuple ( )	Set { }
ინარჩუნებს რიგს	დიახ	დიახ	არა (არ არის გარანტირებული)
ცვალებადია	დიახ	არა	დიახ
დუბლიკატები	დიახ	დიახ	არა
ინდექსები	დიახ	დიახ	არა
slicing	დიახ	დიახ	არა
საერთო ოპერაციები	len(), in, for	len(), in, for	len(), in, for

6.2 მეთოდების მოკლე საცნობარო ცხრილი

ტიპი	მეთოდები	მახასიათებელი
str	lower, upper, capitalize, title, strip, replace, count, split	ყველა აბრუნებს შედეგს; str უცვლელია
list	append, insert, extend, remove, pop, clear, count, index, sort, reverse	მეთოდების უმეტესობა ცვლის თვით მასივს
tuple	count, index	tuple უცვლელია
set	add, update, remove, discard, pop, clear	არ აქვს ინდექსები; მხოლოდ უნიკალური ელემენტები
set ოპერაციები	union, intersection, difference, symmetric_difference	ახალ set-ს აბრუნებს

6.3 სასარგებლო კითხვები დამოუკიდებელი შემოწმებისთვის

- რატომ არ ცვლის name.upper() ცვლადში შენახულ თავდაპირველ ტექსტს?
- რით განსხვავდება split() და strip()?
- რატომ არ უნდა დავწეროთ numbers = numbers.sort()?
- რა განსხვავებაა append([3,4])-სა და extend([3,4])-ს შორის?
- როდის სჯობს tuple მასივს? როდის არის set უკეთესი არჩევანი?
- რას ნიშნავს while-ის პირობის ხელახალი შემოწმება?
- რით განსხვავდება break და continue?

> **გახსოვდეთ:** ყველაზე სასარგებლო პრაქტიკა: თითოეული მაგალითი ჯერ წაიკითხეთ და ივარაუდეთ შედეგი, შემდეგ გაუშვით Python-ში და შეადარეთ. თუ შედეგი განსხვავდება, დააკვირდით ტიპს, არგუმენტებსა და დაბრუნებულ მნიშვნელობას.